CMSC 420: Binary Search Trees

Justin Wyss-Gallifent

January 16, 2026

1 Problems **2**

1 Problems

1. Consider the following tree:

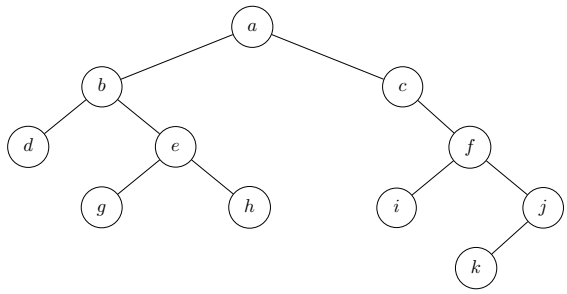


Figure 1: A Binary Search Tree

   (a) List the nodes according to a pre-order traversal.
   (b) List the nodes according to an in-order traversal.
   (c) List the nodes according to a post-order traversal.
   (d) Redraw the tree and include in-order threads.

2. Define b_h to be the number of distinct binary tree structures with height h. We don't care about the contents, just the overall structure.

   (a) What is b_0 and why?

   (b) What is b_1 and why?

   (c) Prove that for $h \geq 2$ we have:

$$b_h = b_{h-1}^2 + 2b_{h-1}\left(1 + b_0 + b_1 + ... + b_{h-2}\right)$$

   (d) Use this to calculate b_h for $n = 2, 3, 4, 5$.

3. A trinary tree is a tree in which each node has between 0 and 3 children (call these the left child, middle child, and right child). Define $T(h)$ for $h \geq -1$ to be the number of distinct trinary tree structures with height h. We don't care about the contents, just the overall structure.

   (a) What is $T(-1)$ and why?

   (b) What is $T(0)$ and why?

   (c) What is $T(1)$ and why?

   (d) Prove that for $h \geq 1$ we have:

   $$T(h) = 3T(h-1)(T(-1)+T(0)+...+T(h-2))^2+3T(h-1)^2(T(-1)+T(0)+...+T(h-2))+T(h-1)^3$$

   (e) Use this to calculate $T(2)$ for $n = 2$.

4. For a binary tree T with n nodes define $d(x, T)$ to be the depth of the node x (the root is depth 0) and define:

$$P(T) = \sum_{x \in T} d(x, T)$$

If T_L and T_R are the two subtrees of the root of a tree T, prove that:

$$P(T) = P(T_L) + P(T_R) + n - 1$$

5. Arrange the numbers 1 through 10 such that when inserted into a binary search tree (which is originally empty) the tree is a complete binary tree after each and every insertion. Briefly explain the logic you used to come up with your answer.

6. (a) Consider the binary tree structure:

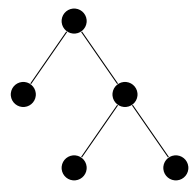


Figure 2: Binary Tree Structure

Assign the list $A = [1, 2, 3, 4, 5]$ as node keys so the result is a binary search tree.

(b) Given an arbitrary binary tree structure with n nodes explain algorithmically (how would you do it?) why it is always possible to select n distinct integers and assign them as node keys within the structure so that the result is a binary search tree.

7. Here is the pseudocode for an inorder traverse:

```
function inorder(x):
    if x != NULL:
        inorder(x.left)
        print(x.key)
        inorder(x.right)
    end if
end function
```

Prove by strong induction on $n \geq 0$, the number of nodes in a tree, that if `inorder(x)` is called where `x` is the root that the output is the keys in increasing order. Assume the keys are all distinct.

8. Define f_h to be the number of distinct full binary tree structures with height h. We don't care about the contents, just the overall structure.

   (a) Draw all possible distinct full binary trees with height $h = 0$ and conclude what the value of f_0 is.

   (b) Draw all possible distinct full binary trees with height $h = 1$ and conclude what the value of f_1 is.

   (c) Draw all possible distinct full binary trees with height $h = 2$ and conclude what the value of f_2 is.

   (d) Prove that for $h \geq 2$ we have:

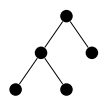
$$f_h = f_{h-1}^2 + 2f_{h-1}\left(f_0 + f_1 + ... + f_{h-2}\right)$$

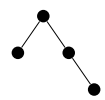
9. For a binary tree T containing n nodes there are ways to measure how balanced it is with a single number. One of them is the fill ratio:

$$FR(T) = \frac{n}{2^{H(T)+1} - 1}$$

The fill ratio essentially asks - what is the ratio of the current number of nodes to the number of nodes which would be in the tree if it had the same height and was perfect (full on every level).

(a) Calculate this value for the following trees:

i. 

ii. 

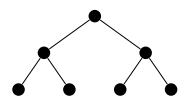
iii. 

Figure 3: Fill Ratio Trees

(b) For a binary tree with n nodes what are the maximum and minimum fill ratios? Calculate these for each of the values $n = 1, ..., 10$, approximating to four digits after the decimal point for each.

(c) For a binary tree with height h what are the maximum and minimum fill ratios? Calculate these for each of the height $h = 1, ..., 10$, approximating to four digits after the decimal point for each.

10. Suppose a binary tree contains n nodes.

    (a) Write down an expression for the minimum possible height of the tree, with justification.

    (b) Write down an expression for the maximum possible height of the tree, with justification.

11. Draw a single binary tree which has the following two properties:

- For every node x we have $\text{height}(x.left) - \text{height}(x.right) \in \{-1, 0, 1\}$.
- There are two leaves whose levels differ by 2.

12. A binary search tree has the following structure:

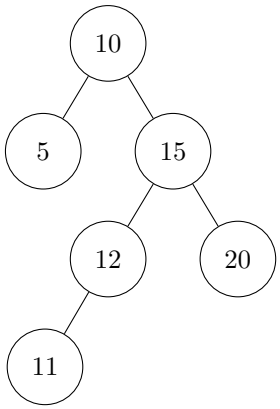


Figure 4: A Binary Search Tree

Suppose we delete a key and the height of the tree changes. Various choices of key and/or use of IOP/IOS replacements could make this happen. What are the possibilities? Explain each.

13. Suppose we have a binary search tree containing n nodes. We pick a key at random with all keys equally likely and delete it. Suppose that deleting a node on level k takes time exactly k (accounting for all necessary work).

    What structure could the tree have which would guarantee that the average-case time to do this would be $\Theta(n)$? Justify.

14. Suppose we have a perfect binary search tree with L levels indexed $0, ..., L-1$. Thus it has $n = 2^L - 1$ nodes. Suppose that deleting a node on level k takes time exactly k (accounting for all necessary work).

    (a) For an unknown $0 \leq k \leq L-1$ what is the probability that we pick a key on level k?

    (b) We pick a key at random with all keys equally likely and delete it. How long do we expect it to take to delete the key?
    Hint: Sum over the levels, and you may need to find a formula for this sum.

15. Suppose a binary tree has the property that each node's key is larger than the key in its left child (if one exists) and smaller than the key in its right child (if one exist). Is this same as being a BST? Explain.

16. Consider this binary search tree:

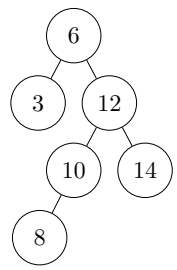


Figure 5: A Binary Search Tree

The following are separate questions about this tree.

(a) We run `search(x)` where x is a key in the tree chosen randomly with all keys equally likely. If it takes time $k+1$ to find a key on level k how much time do we expect the search to take? Simplify.

Expected Time	

(b) We run `insert(x)` where x is an integer, $1 \leq x \leq 15$, and x differs from all keys in the tree. The height of the tree does not change. What could x be? List all values in increasing order.

Possible x	

(c) Suppose we delete a key and the height of the tree does not change. Various choices of key and/or use of IOP/IOS replacements could make this happen. What are the possibilities? Explain each. You don't need pictures, just list the keys which could be deleted and, if a replacement is used, what that would be and whether it is the IOP or IOS.

17. Consider all binary trees which have the property that for every node the height of the left and right subtrees differ by at most 2. Denote by N_h the number of different trees with this property. We don't care about the contents, just the overall structure.

(a) Show that $N_2 \geq 3$ by drawing exactly three such trees.

(b) Consider the following possibilities:

(A) $N_h = N_{h-1}^2 + N_{h-1}N_{h-2} + N_{h-1}N_{h-3}$
(B) $N_h = N_{h-1}^2 + 2N_{h-1}N_{h-2} + 2N_{h-1}N_{h-3}$
(C) $N_h = 2N_{h-1}N_{h-1} + 2N_{h-1}N_{h-2} + 2N_{h-1}N_{h-3}$
(D) None of the above.

Which of these is true?

Which? A or B or C or D?	